

Architecture.md v1.1

Chapter 13 — Project Folder Structure & Code Organization

This chapter defines the official directory structure of AI OS. Every developer, automation tool, and AI agent must follow this structure.

No component may create arbitrary folders outside this specification without an approved architecture update.

13.1 Design Philosophy

The project structure is organized by responsibility rather than by technology.

Objectives:

- Easy navigation
 - High modularity
 - Clear ownership
 - Independent testing
 - Future scalability
-

13.2 Root Directory

```
AI_OS/  
├─ docs/  
├─ backend/  
├─ frontend/  
├─ agents/  
├─ core/  
├─ memory/  
├─ plugins/  
├─ config/  
├─ database/  
├─ logs/  
└─ scripts/
```

```
|─ tests/
|─ assets/
|─ backups/
|─ temp/
|─ requirements.txt
|─ .env.example
|─ README.md
└─ LICENSE
```

13.3 docs/

Contains all project documentation.

```
docs/

Vision.md
Architecture.md
Development_Guide.md
Roadmap.md
API.md
CHANGELOG.md
```

No source code belongs here.

13.4 backend/

Contains server-side application logic.

```
backend/

api/
router/
services/
controllers/
middleware/
utils/
```

Responsibilities:

- REST APIs

- Internal services
 - Request handling
 - Middleware
-

13.5 frontend/

Contains the user interface.

Future versions may include:

- Desktop Dashboard
- Web Dashboard
- Mobile Interface

The frontend communicates only with backend APIs.

13.6 agents/

Contains every Worker Agent.

```
agents/  
  
jarvis/  
friday/  
  
planner/  
analytics/  
business/  
finance/  
research/  
content/  
browser/  
calendar/  
email/  
vision/  
automation/  
trading/  
pc_control/
```

Each agent is an independent module.

13.7 Standard Agent Structure

Every agent follows the same layout.

```
agent_name/  
  
agent.py  
config.py  
schemas.py  
models.py  
permissions.py  
tests.py  
README.md
```

This structure is mandatory.

13.8 core/

Contains shared system components.

```
core/  
  
agent_manager/  
router/  
permission_service/  
logging_service/  
event_service/  
workflow_engine/  
plugin_manager/  
monitoring/
```

Business logic does not belong in this directory.

13.9 memory/

Contains the Memory Service.

```
memory/
```

```
memory_service.py
memory_manager.py
embedding_service.py
cache.py
schemas.py
```

Future semantic memory modules will also reside here.

13.10 plugins/

Contains optional or external plugins.

Examples:

```
plugins/

weather/
music/
stocks/
discord/
slack/
```

Plugins must register through the Plugin Manager.

13.11 config/

Contains configuration files.

```
config/

config.yaml
permissions.yaml
models.yaml
logging.yaml
```

Secrets are never stored here.

13.12 database/

Contains persistence-related files.

```
database/  
  
migrations/  
repositories/  
sqlite/  
postgres/
```

Only the Memory Service interacts with this directory.

13.13 logs/

Stores runtime logs.

Subdirectories:

```
logs/  
  
system/  
agents/  
security/  
errors/  
audit/
```

Logs must never be manually edited.

13.14 scripts/

Contains automation utilities.

Examples:

- Database migration
- Backup creation
- Project setup
- Maintenance scripts

13.15 tests/

Contains automated tests.

```
tests/  
  
unit/  
integration/  
system/  
performance/
```

Every major module should have corresponding tests.

13.16 assets/

Stores non-code resources.

Examples:

- Icons
- Images
- Audio
- UI resources

13.17 backups/

Contains system backups.

Examples:

- Database backups
- Configuration backups
- Memory snapshots

Backups are read-only during normal operation.

13.18 temp/

Stores temporary runtime files.

Examples:

- Downloads
- Cached files
- Temporary exports

This directory may be cleared automatically.

13.19 Naming Conventions

Directories:

- `snake_case`

Files:

- `snake_case.py`

Classes:

- `PascalCase`

Functions:

- `snake_case()`

Constants:

- `UPPER_SNAKE_CASE`
-

13.20 Import Rules

Allowed:

```
Agent  
|
```

▼
Core Services

Forbidden:

Analytics Agent
|
▼
Business Agent

Worker Agents must never import one another directly.

13.21 Design Rules

The project structure must satisfy:

- Clear separation of responsibilities.
- Modular organization.
- Consistent naming.
- Independent agent development.
- Easy automation.
- Scalable expansion.

Any structural modification requires an update to Architecture.md.

End of Chapter 13

Next Chapter: Chapter 14 — Development Contracts & Coding Standards